AUTOMATED REVIEW RATING SYSTEM

IMBALANCED DATASET

Jeevan Santhosh S

PROJECT OVERVIEW

An intelligent system that analyzes customer reviews and predicts corresponding star ratings. It uses natural language processing (NLP) to extract sentiment and key features from textual feedback. The model is trained on labelled review data to learn patterns between language and rating.

ENVIRONMENT SETUP

- Installed Python 3.10+ with required libraries: pandas, numpy, matplotlib, seaborn, spacy, scikit-learn, beautifulsoup4, requests.

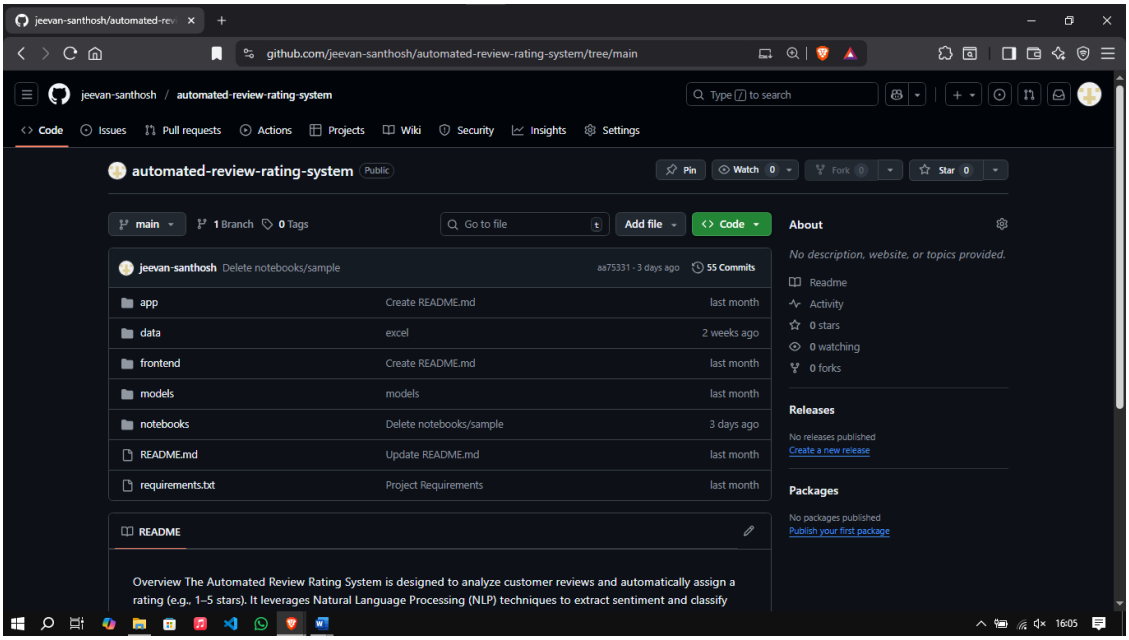
- IDE used: VS Code

- Verified proper functioning of packages for data preprocessing, NLP, and visualization.

GITHUB PROJECT SETUP

The project was set up on GitHub, a new repository was created to store datasets, preprocessing scripts, and model files. The setup process included initializing the repository, configuring Git on the local system, and pushing the project files to GitHub. A screenshot of the GitHub repository setup is included below to demonstrate the successful configuration and repository structure for the Automated Review Rating System. Link of GitHub repository : **automated-review-rating-system.**

Structure of the directory :



DATA COLLECTION

- 3 Datasets were collected through Kaggle.
- Datasets collected from Kaggle when combined had 2,01,538 rows and these were datasets related with flipkart reviews.
- Dataset link – Expanded Dataset

DATA PREPROCESSING

Effective data preprocessing is essential to improve the performance and accuracy of machine learning models. The following techniques were applied to prepare the raw review dataset for modeling.

1. Removing Duplicates

Duplicate rows containing the exact same review and rating were removed to prevent bias and overfitting. This ensured that the dataset only included unique observations.

Code :

```
expanded_dataset.drop_duplicates(subset=["Review"], inplace=True)
```

2. Removing Conflicting Reviews

Some reviews had identical text but different star ratings. These inconsistencies can confuse the model. Such conflicting entries were identified and removed to maintain label clarity.

Code :

```
conflict_reviews =
expanded_dataset.groupby("Review")["Rating"].nunique()
conflict_reviews = conflict_reviews[conflict_reviews > 1].index

expanded_dataset =
expanded_dataset[~expanded_dataset["Review"].isin(conflict_reviews)]
```

3. Handling Missing Values

Rows with missing or null values-particularly in the review text or rating column-were removed. This step ensured the dataset was complete and meaningful for analysis.

Code :

```
expanded_dataset.dropna(subset=["Review", "Rating"], inplace=True)
```

4. Dropping Unnecessary Columns

Non-essential columns such as review IDs, timestamps, or user identifiers were dropped. These fields did not contribute to the model and could introduce noise or privacy concerns. Rows with missing or null values particularly in the review text or rating column were removed. This step ensured the dataset was complete and meaningful for analysis

Code:

```
expanded_dataset = expanded_dataset[["ID", "Rating", "Review"]].copy()
```

5. Lowercasing Text

All review text was converted to lowercase to maintain uniformity. This helps prevent duplication of tokens like "Good" and "good" being treated as separate words.

Code :

```
text = str(text).lower()
```

6. Removing URLs

URLs present in the review text were removed using regular expressions.

Code :

```
text = re.sub(r'http\S+|www.\S+', '', text)
```

7. Removing Emojis and Special Characters

Emojis and special symbols may not contribute meaningful context for text analysis (unless specifically required). We remove these characters to focus on the core textual content.

Code :

```
emoji_pattern = re.compile("["
                                u"\U0001F600-\U0001F64F"
                                u"\U0001F300-\U0001F5FF"
                                u"\U0001F680-\U0001F6FF"
                                u"\U0001F1E0-\U0001F1FF"
```

```
                                "]+", flags=re.UNICODE)
    text = emoji_pattern.sub(r'', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

8. Removing Puncuations

Punctuation marks and numbers can disrupt the natural language patterns of the text. By removing then, we simplify the tokenization process and prevent punctuation from being misinterpreted as separate tokens.

Code :

```
text = re.sub(r'<.*?>', '', text)
```

9. Stopwords Removal

Stopwords, such as "the", "is", and "and", are frequently occurring words that might not add significant semantic value. We use libraries like Spacy to filter these words out, reducing noise and focusing on words that contribute meaning to the reviews, there were total 175 stop words in spacy.

```
("under", "it", "for", "is", "but", "to", "its", "very", "this", "with",
"so", "if", "any", "has", "that", "s", "of", "can", "only", "i", "you",
"are", "or", "my", "am", "over", "all", "a", "not", "in", "was", "and",
"few", "other", "same", "about", "these", "doesn", "t", "m", "ll",
"again", "after", "as", "now", "the", "because", "doing", "more", "than",
"just", "don", "have", "at", "when", "up", "some", "will", "who", "no",
"on", "didn", "why", "from", "re", "they", "here", "be", "while",
"there", "which", "we", "should", "what", "between", "too", "once", "me",
"your", "then", "our", "been", "how", "an", "off", "d", "o", "during",
"nor", "their", "y", "won", "ve", "above", "both", "having", "below",
"such", "most", "down", "being", "until", "each", "couldn", "out",
"into", "before", "through", "further", "yours", "her", "she", "he",
"had", "them", "himself", "where", "those", "them", "own", "myself",
"itself", "yourselves", "themselves", "haven", "isn", "aren", "wouldn",
"shouldn", "hadn", "does", "did", "doing", "done", "having")
```

Code:

```
import nltk
nltk.download('stopwords')
nltk.download('punkt')
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import spacy

nlp = spacy.load('en_core_web_sm')
nltk_stopwords = set(stopwords.words('english'))
```

```
def preprocess_text(text):
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if token.text not in
nltk_stopwords and token.is_alpha]
    return " ".join(tokens)
```

10. Lemmatization

Lemmatization is a text preprocessing technique that reduces words to their base or dictionary form, known as the lemma. For example, words like "running", "ran", and "runs" are all reduced to the base form "run". Unlike stemming, lemmatization uses linguistic rules and vocabulary, ensuring that the resulting word is meaningful. This helps in grouping similar words and improves model performance by reducing the size of the vocabulary without losing context.

Why lemmatization is better than stemming?

Lemmatization is often preferred over stemming because:

1. Produces Real Words:
Lemmatization returns valid dictionary words (e.g., "running" "run"), whereas stemming may return incomplete or non-existent words (e.g., "running" → "runn").

2. Context-Aware:
Lemmatization considers the context and part of speech (POS) of a word to find its correct root form.
Stemming blindly trims word endings without understanding grammar.

3. More Accurate and Clean:
Lemmatization provides cleaner and more accurate tokens, which improves model understanding and performance, especially in NLP tasks like sentiment analysis or classification.

Code :

```
def preprocess_text(text):
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if token.text not in
nltk_stopwords and token.is_alpha]
    return " ".join(tokens)
```

11. Filtering by Word Count

Very short reviews might not contain enough context to be useful, and excessively long reviews could be outliers. We apply filtering to exclude:

- Reviews with fewer than 3 words.
- Reviews that exceed 100 words.
  This ensures that the dataset remains robust and relevant for model training.

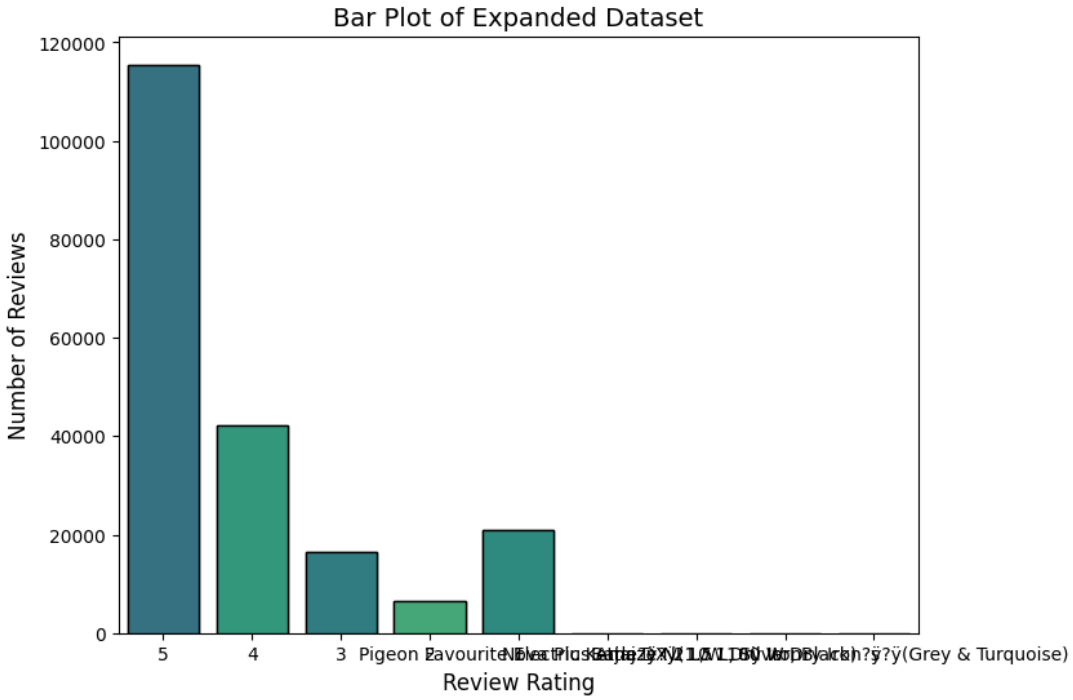
Code :

```
before_word_filter = len(expanded_dataset)

expanded_dataset = expanded_dataset[expanded_dataset["Review"].apply
                          (lambda x: 3 <= len(str(x).split()) <=
                          100)]

after_word_filter = len(expanded_dataset)
```

DATA VISUALIZATION

Bar Plot

A bar plot is a chart that represents categorical data with rectangular bars, where the height of each bar indicates the value or frequency of that category. In this project, bar plots were used to show the number of reviews per rating class (1 to 5 stars), helping to visualize class distribution and detect any imbalance in the dataset.



IMBALANCED DATASET

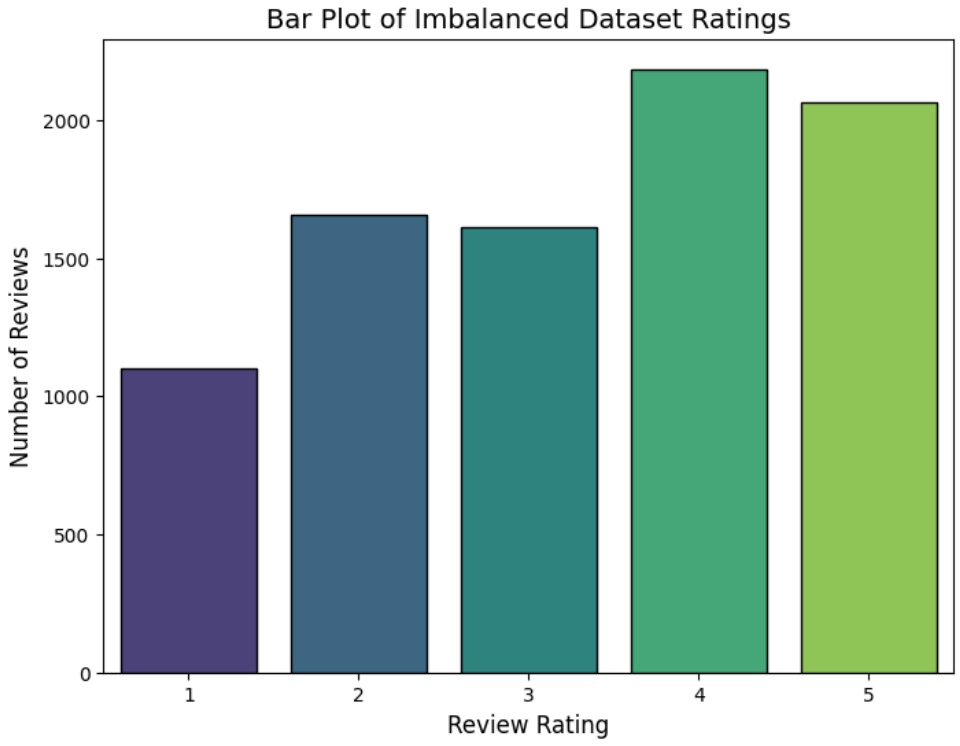
To ensure that the model is trained on a biased dataset, an imbalanced dataset was created by selecting reviews in a random distribution for each rating from 1 to 5. It was in the form : **1 star – 10%; 2 star – 50%; 3 star – 25%; 4 star – 15% & 5 star – 5%**. This resulted in a **total of 8623 reviews**, ensuring unequal representation across all rating categories. Link of Imbalanced Dataset :

Code :

```
target_percentages = {
    1: 0.10,  # 10% of available Rating 1 reviews
    2: 0.50,  # 50% of available Rating 2 reviews
    3: 0.25,  # 25% of available Rating 3 reviews
    4: 0.15,  # 15% of available Rating 4 reviews
    5: 0.05   # 05% of available Rating 5 reviews
}

# Total available count per rating in the cleaned data
available_counts = df_cleaned['Rating'].value_counts().sort_index()
print("\nAvailable reviews per rating after cleaning (to check
feasibility):\n")
print(available_counts)
```

1. Data Visualization of Imalanced Dataset

Bar Plot of Imalanced Dataset :



2. Sample Reviews of Imbalanced Dataset

- 1 STAR (5 reviews)
    1. month left minutes temparature
    2. small size use outside bathroom
    3. speed air delivery totally disappointed expect lot company renesa model low rpm air delivery
    4. machine cheap quality time use working blades made plastic nd jare fitting grinder
    5. bought days working

- 2 STAR (5 reviews)
    1. genuine product timex qr code work timex app strap hard leather
    2. vary low quality
    3. box backing work bad
    4. motor ok heating problemits takes much time grind
    5. average product cushions rise properly edges remain hollow

- 3 STAR (5 reviews)
    1. nice product surfing mobile network broadband needed installation little hard
    2. cheap bad cushion
    3. looks good bt washing time color going
    4. nice products superb quality value money thanks flipkart
    5. average tv without ips display viewing angles bad plus chromecast works like charm better go sony samsung mitv cannot match former picture quality recommended looking functional tv budget

- 4 STAR (5 reviews)

    1. ossmm product good qualitybetter packaging
    2. think best product prize totally worth get bag memory card gave star
    3. looks elegant classy mat finish colour definitely high speed fan little bit noisy fan
    4. awesome product fabric quality
    5. took product months ago best price k would early judge touch wood far extremely happy product flip kart really kind deliver product time followed quick demo product brief recommend product

- 5 STAR (5 reviews)

    1. good installation demo mr pranav pawar jeeves tech
    2. first ever apple product love already interface smooth beautiful also apple takes security concerns seriously adds even professional touch flaw found far sub par camera also thanks flipkart delivering exact product time additional note advisable buy ipad th gen rather th one since get much much price
    3. yes happy oder big billion offer get price original product really heavy really happy product flipkart thank much
    4. original good quality
    5. good nad well packed nice product

TRAIN TEST SPLIT

Train-Test Split is a technique to divide the dataset into two separate sets:

- Training Set (typically 80%): Used to train the machine learning model.
- Test Set (typically 20%): Used to evaluate the model's performance on unseen data.

This separation ensures that the model generalizes well and is not just memorizing the training data.

Why Stratified Split?

In classification problems like review rating prediction, stratified splitting is important to maintain class balance (equal distribution of ratings) in both training and testing sets.

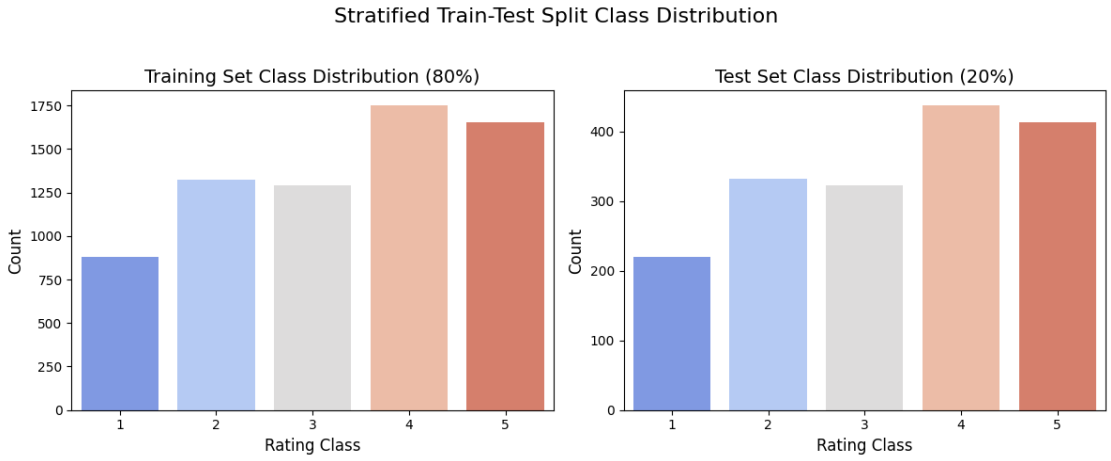
Without stratification, some rating classes (like 1-star or 5-star) may be underrepresented in the test set, leading to biased evaluation.

Code :

```
# Stratified split (80% train, 20% test)
X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y,
    test_size=0.2,
    stratify=y, # Ensures the class proportion is maintained in both splits
    random_state=42
```

How It Was Done:

To prepare the data for model training, the dataset was first shuffled randomly to eliminate any order bias. A stratified train-test split was then performed using train teat split from rkimarn.model selection with the ratify-y argument to ensure that all star ratings were proportionally represented in both training and testing sets. The dataset was split using an 80% training and 20% testing ratio. After splitting, all text preprocessing stops-Including lowercasing. Iemmatization, stopword removal, and cleaning were applied separately tox train and x text to prevent data leakage and maintain model integrity.



TEXT VECTORIZATION

Machine learning models can't work directly with raw text-they require numerical input. Vectorization is the process of converting text data into numerical features.

TF-IDF (Term Frequency - Inverse Document Frequency)

TF-IDF is a statistical technique that represents how important a word is to a document relative to the entire corpus.

- TF (Term Frequency): How often a word appears in a document.
- IDF (Inverse Document Frequency): Penalizes common words and highlights rare but Important ones.

This approach helps to capture both word relevance and discriminative power, making it better than simple word counts.

Formula for TF-IDF:

$$TF(t,d) = \frac{number\ of\ times\ t\ appears\ in\ d}{total\ number\ of\ terms\ in\ d}$$

$$IDF(t) = log\frac{N}{1+df}$$

$$TF-IDF(t,d) = TF(t,d) * IDF(t)$$

Code :

```
tfidf = TfidfVectorizer(stop_words='english', max_features=10000,
ngram_range=(1, 2))
X_train = tfidf.fit_transform(X_train_text)
X_test = tfidf.transform(X_test_text)
```

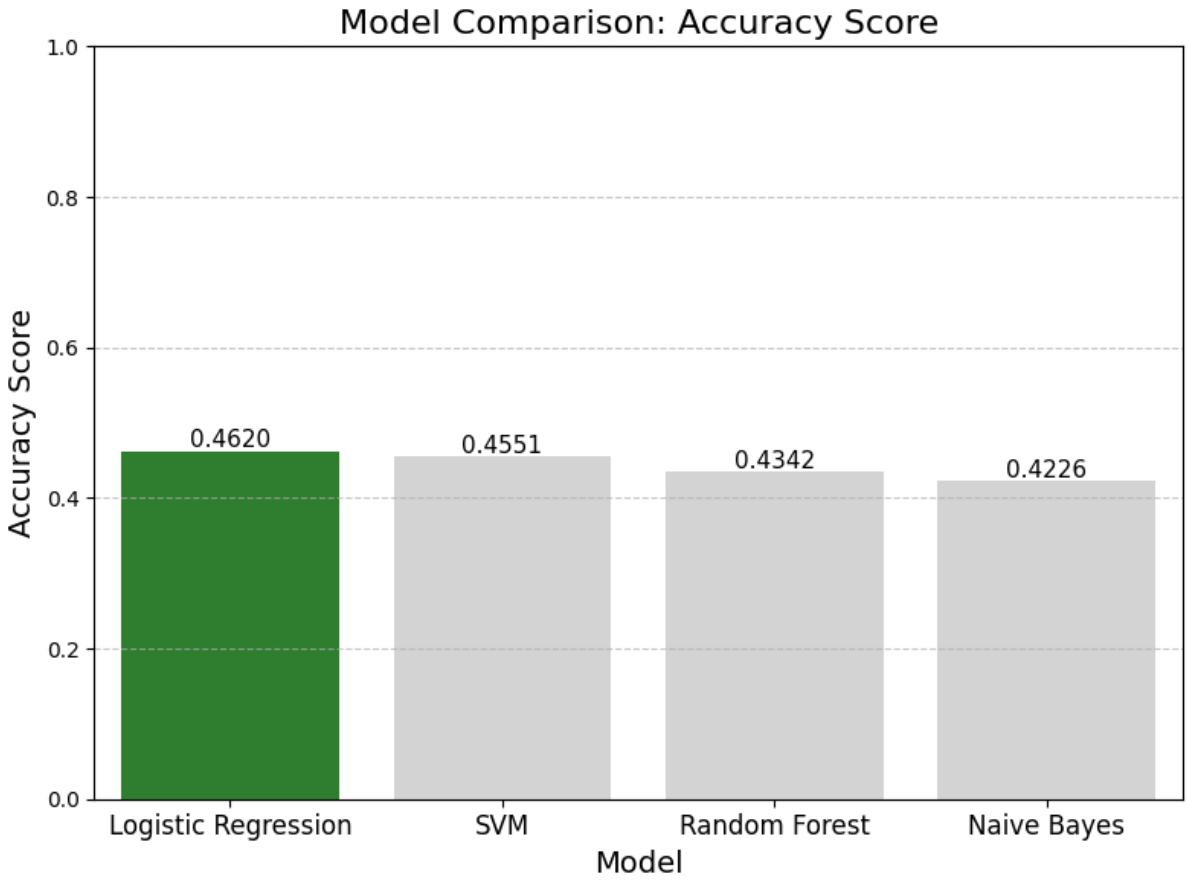
MODEL PERFORMANCE ON TRAINING

The evaluation of multiple machine learning models trained on a imbalanced text review dataset to predict sentiment or rating categories. The imbalanced dataset ensures unequal representation of all classes, allowing a unfair comparison of model performance with bias toward dominant classes. Four models are Logistic Regression, Naive Bayes, Random

Forest, and SVM were assessed using key metrics such as accuracy, precision, recall, and F1-score to determine their effectiveness and generalization capability for text classification tasks.

MODEL	ACCURACY	PRECISION	RECALL	F1-SCORE
Logistic Regression	0.4620	0.56	0.45	0.49
Naive Bayes	0.4226	0.65	0.20	0.31
Random Forest	0.4342	0.52	0.37	0.43
SVM	0.4551	0.60	0.38	0.47

Among all models trained on the imbalanced dataset, **Logistic Regression** achieved the highest overall performance with an accuracy of **0.4620**, followed closely by SVM at 0.4551. Both models demonstrated strong precision-recall balance, indicating effective generalization across all rating classes. Tree-based models like Random Forest showed lower performance, suggesting that linear models are better suited for the TF-IDF-based text representation used in this dataset.



CROSS-TEST (Imbalanced - Balanced)

When the model trained on the imbalanced dataset was tested on the balanced dataset, it achieved a **cross-test accuracy of 0.5686**, with a **weighted precision of 0.7664** and a **weighted F1 score of 0.6231**. These results indicate that the model generalizes reasonably well to a different data distribution, maintaining strong overall performance despite class imbalance.

Which model would you deploy and why?

Based on the evaluation and fine-tuning results, Logistic Regression is the recommended model for deployment. It achieved the highest accuracy (0.4620) on the imbalanced dataset, maintained strong performance during cross-testing on the balanced dataset (accuracy 0.5686), and demonstrated a good balance of precision and F1-score. Logistic Regression is well-suited for high-dimensional TF-IDF text features, providing robust generalization across classes. Its linear decision boundary ensures efficient computation and scalability, making it the best choice for reliable, production-ready text classification in this scenario.

UI & DEPLOYMENT

For UI creation I used Streamlit. It provides an interactive interface to analyze customer reviews using deep learning models. Users can input any review text, and the app instantly displays predicted ratings from two trained models one trained on balanced data and another on imbalanced data. It visually compares their outputs, confidence scores, and overall reliability. This prototype helps demonstrate how dataset balance affects model performance and prediction stability, offering a user-friendly way to test and evaluate review sentiment classification in real time.